

Zustand

Shared Client State Made Simple

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Distinguish server state, UI state, and shared client state
- ▶ Create a Zustand store and read state with selectors
- ▶ Avoid re-render traps with `useShallow`
- ▶ Add persistence and DevTools with middleware
- ▶ Know when to use a store and when not to

Section Items

- ▶ Two Kinds of State
- ▶ Creating a Store
- ▶ Killing Prop Drilling
- ▶ Selectors and `useShallow`
- ▶ Middleware: `persist` and `devtools`
- ▶ When to Use a Store

Section Items

► Two Kinds of State

- Creating a Store
- Killing Prop Drilling
- Selectors and `useShallow`
- Middleware: `persist` and `devtools`
- When to Use a Store

Two Kinds of State

- ▶ **Server state** — lives on the backend. Cache: TanStack Query.
- ▶ **Client state** — lives in the browser. Two flavors:
 - ▶ **UI state** — modal open?, current tab, form input. Owner: a component. Tool: `useState`.
 - ▶ **Shared client state** — cart contents, current user, app theme, filter applied app-wide. Multiple components read AND write. Tool: a **store**.
- ▶ Today: shared client state with **Zustand**.

Why a Store? Why Not Context?

Context works. But:

- ▶ Every consumer re-renders whenever **any** value in the provider changes
- ▶ You end up splitting Context into 5 providers to keep re-renders surgical → boilerplate
- ▶ No DevTools. No middleware. No subscribe-outside-React.

A store gives you selectors (subscribe to a slice) for free.

Why Zustand vs Redux Toolkit?

	Redux Toolkit	Zustand
Setup	createSlice, reducers, actions, Provider	one create() call
Boilerplate	lots	almost none
DevTools	yes	yes (middleware)
Async	thunks, RTK Query	just async a function
Industry weight	huge	growing fast

Same architecture, 1/5 the code. Redux is still the right answer for a 500k-line codebase.

Section Items

- Two Kinds of State

► **Creating a Store**

- Killing Prop Drilling
- Selectors and `useShallow`
- Middleware: `persist` and `devtools`
- When to Use a Store

The Shape of a Store

```
import { create } from "zustand";

interface TodoStore {
  todos: Todo[];
  addTodo: (text: string, minutes: number) => void;
}

export const useTodoStore = create<TodoStore>((set) => ({
  todos: DEFAULT_TODOS,
  addTodo: (text, minutes) =>
    set((state) => ({
      todos: [...state.todos, { id: Date.now(), text, done: false,
        minutes }],
    })),
})));
```

Reading State

```
// Pick one slice. Re-renders only when THIS slice changes.  
const todos = useTodoStore((s) => s.todos);  
const addTodo = useTodoStore((s) => s.addTodo);
```

- ▶ Each call subscribes to one slice of state
- ▶ Re-renders only when that slice changes — not the whole store

Using the Store Outside React

```
useTodoStore.getState().todos;  
useTodoStore.getState().addTodo("hi", 5);  
useTodoStore.subscribe((s) => console.log(s.todos));
```

- ▶ A store is just a JS object with a hook attached
- ▶ Access state from non-component code: logging, tests, utilities

Live Code: Minimal Store

- ▶ Open `store/todoStore.ts` — the entire store in one file
- ▶ Open `App.tsx` — no props passed, components pull from the store

Section Items

- Two Kinds of State
- Creating a Store
- ▶ **Killing Prop Drilling**
 - Selectors and `useShallow`
 - Middleware: `persist` and `devtools`
 - When to Use a Store

Before: Props Everywhere

```
<TodoList
  todos={visible}
  onToggle={toggleTodo}
  onDelete={deleteTodo}
/>
```

Every callback threaded from parent to child to grandchild.

After: Components Pull What They Need

```
<TodoList todos={visible} />
```

Inside TodoCard:

```
const toggleTodo = useTodoStore((s) => s.toggleTodo);
```

- ▶ Each component pulls what it needs — the tree gets flatter
- ▶ No more threading callbacks through 3 layers of props

Action References Are Stable

- ▶ Zustand never recreates action functions
- ▶ Subscribing to an action does NOT cause re-renders
- ▶ Cheap to pull actions anywhere — no `useCallback` needed

When NOT to Use the Store

- ▶ Filter state in `<TodoFilter>` stays local (`useState`)
- ▶ Rule: lift to the lowest common ancestor that needs it
- ▶ Not everything belongs in the store — be picky

Section Items

- Two Kinds of State
- Creating a Store
- Killing Prop Drilling
- ▶ **Selectors and** `useShallow`
 - Middleware: `persist` and `devtools`
 - When to Use a Store

The Basic Selector

```
const count = useTodoStore((s) => s.todos.length);
```

- ▶ Returns a primitive — re-renders only when `length` changes
- ▶ Default `Object.is` compare is enough for primitives

The Trap: New Object Every Call

```
const { active, done } = useTodoStore((s) => ({
  active: s.todos.filter((t) => !t.done).length,
  done: s.todos.filter((t) => t.done).length,
}));
```

- ▶ Returns a **new object** every call
- ▶ `Object.is(prevObj, newObj) === false` always
- ▶ Re-renders on **every** store change, even unrelated ones

The Fix: useShallow

```
import { useShallow } from "zustand/react/shallow";

const { active, done } = useTodoStore(
  useShallow((s) => ({
    active: s.todos.filter((t) => !t.done).length,
    done: s.todos.filter((t) => t.done).length,
  }))
);
```

useShallow switches the compare to shallow-equality on object keys.

Parameterized Selectors

```
export function useVisibleTodos(filter: Filter) {  
  return useTodoStore(  
    useShallow((s) => s.todos.filter(matches(filter)))  
  );  
}
```

- ▶ `filter` is a closure variable
- ▶ Re-evaluates when `filter` changes
- ▶ Encapsulates the filtering rule

The Array Trap

```
// WRONG -- infinite loop  
useTodoStore((s) => s.todos.filter(...))
```

- ▶ `.filter()` returns a **new array reference** every call
- ▶ `Object.is(prev, next)` is always false
- ▶ React re-renders → selector runs → new array → re-render → boom
- ▶ Browser shows: Maximum update depth exceeded

Rule: any selector that returns an object OR an array needs `useShallow`. Primitives are safe without it.

- ▶ Remove `useShallow` from a selector that returns an array
- ▶ Watch the browser crash with “Maximum update depth exceeded”
- ▶ Add it back — fixed

Section Items

- Two Kinds of State
- Creating a Store
- Killing Prop Drilling
- Selectors and useShallow
- ▶ **Middleware:** persist **and** devtools
- When to Use a Store

What Is Middleware?

```
create<T>()(  
  devtools(  
    persist(  
      (set) => ({ ...storeBody }),  
      { name: "zustand-demo.todos" }  
    ),  
    { name: "TodoStore" }  
  )  
);
```

A function that wraps the store factory. Composes inside-out: `persist` wraps the body, `devtools` wraps the wrapped.

persist

```
persist(  
  (set) => ({ ...storeBody } ),  
  {  
    name: "zustand-demo.todos",  
    partialize: (s) => ({ todos: s.todos } ),  
    version: 1,  
  }  
)
```

- ▶ Syncs to localStorage on every set()
- ▶ Rehydrates on startup before the first render
- ▶ partialize — opt-in which keys to save (skip action functions)
- ▶ version — bump to invalidate old saved data

persist Replaces the useEffect Pattern

The React snapshot had this in a custom hook:

- ▶ `useEffect(() => { localStorage.setItem(KEY, JSON.stringify(todos)); }, [todos]);`
- ▶ `persist` middleware replaces it entirely
- ▶ No more “did I sync to localStorage?” — the middleware always does

- ▶ Records every `set()` as a Redux DevTools action
- ▶ Time-travel: rewind the UI by clicking through history
- ▶ Zero cost in production (auto-disabled when `NODE_ENV === "production"`)

Live Code: Middleware

- ▶ Add a todo, refresh — it persists (check Application → Local Storage)
- ▶ Open Redux DevTools — see every action logged
- ▶ Click through the history — watch the UI rewind

Section Items

- Two Kinds of State
- Creating a Store
- Killing Prop Drilling
- Selectors and `useShallow`
- Middleware: `persist` and `devtools`

► **When to Use a Store**

Use a Store When

- ▶ Two+ components both read AND write the same state, and they're not parent-child
- ▶ You're passing the same callback down 3+ levels
- ▶ You need state outside React (e.g. logging from a non-component module)

Don't Use a Store For

- ▶ Form input values (local `useState`)
- ▶ Modal open/closed for one modal (parent's `useState`)
- ▶ Server data — that's TanStack Query (caching, invalidation, staleness are a different problem)

Anti-pattern: “Put everything in Zustand.” You'll lose surgical re-renders and the readability of co-located state. Be picky.

Industry Landscape (Awareness Only)

Library	One-liner
Redux Toolkit	Incumbent. Resume keyword. More boilerplate.
Jotai	Atom-based. Closer to Recoil.
Valtio	Proxy-based. Mutate directly.
MobX	Observable-based. Older.
TanStack Query	Server state only. Pairs with Zustand.

Zustand is the modern default for client state. Redux is the resume-driven default.

What You Can Do Now

- ▶ Create a Zustand store with typed state and actions ✓
- ▶ Read state with selectors — surgical re-renders ✓
- ▶ Use `useShallow` for object/array returns ✓
- ▶ Add `persist` and `devtools` middleware ✓
- ▶ Know when a store is the right tool and when it's not ✓

The Mental Model

- ▶ A store = a hook + state outside React
- ▶ Selectors = surgical re-renders
- ▶ `useShallow` for object/array returns
- ▶ Middleware = features without touching components
- ▶ Use stores for **shared client state**; use TanStack Query for **server state**